Wireless

```
import simpy
import random

random.seed(1)

env = simpy.Environment()
channel = simpy.Store(env)

stats = {"received": 0, "lost": 0, "delays": []}


def client(env, src, dst, tech, distance, n, interval):
    settings = {
        "Wi-Fi": (100, 0.02, 0.005, 0.020),
        "Bluetooth": (10, 0.05, 0.010, 0.050),
        "Zigbee": (50, 0.03, 0.020, 0.100)
    }

    max_range, loss, min_delay, max_delay = settings[tech]

    for i in range(1, n + 1):
        yield env.timeout(interval)

        p = {
            "id": i,
            "src": src,
            "dst": dst,
            "tech": tech,
            "distance": distance,
            "range": max_range,
            "loss": loss,
            "delay": random.uniform(min_delay, max_delay),
            "time": env.now
        }

        print(src, "sends Packet", i, "using", tech)
        yield channel.put(p)


def server(env, channel, stats):
    while True:
        p = yield channel.get()

        if p["distance"] > p["range"]:
            print("Packet", p["id"], "dropped: Out of range")
            stats["lost"] += 1

        elif random.random() < p["loss"]:
            print("Packet", p["id"], "lost")
            stats["lost"] += 1

        else:
            yield env.timeout(p["delay"])

            delay = env.now - p["time"]
            stats["received"] += 1
            stats["delays"].append(delay)

            print("Server received Packet", p["id"])
            print("Technology:", p["tech"])
            print("Delay:", round(delay * 1000, 2), "ms")


env.process(server(env, channel, stats))

env.process(client(env, "W1", "W2", "Wi-Fi", 20, 5, 0.2))
env.process(client(env, "B1", "B2", "Bluetooth", 5, 5, 0.5))
env.process(client(env, "Z1", "Z2", "Zigbee", 10, 5, 1))

env.run()

print("\n--- FINAL RESULTS ---")
print("Packets generated:", 15)
print("Packets received:", stats["received"])
print("Packets lost:", stats["lost"])

if stats["delays"]:
    avg = sum(stats["delays"]) / len(stats["delays"])
```

```
    print("Average delay:", round(avg * 1000, 2), "ms")

print("Delivery ratio:",
      round(stats["received"] / 15 * 100, 2), "%")
```

Routing

```
import simpy

routes = {
    "R1": {
        "192.168.2.0/24": "R2",
        "192.168.3.0/24": "R2"
    },
    "R2": {
        "192.168.1.0/24": "R1",
        "192.168.3.0/24": "R3"
    },
    "R3": {
        "192.168.1.0/24": "R2",
        "192.168.2.0/24": "R2"
    }
}


def server(env, channel):
    while True:
        packet = yield channel.get()

        path = []
        router = packet["router"]
        dst = packet["dst"]

        while router:
            yield env.timeout(0.01)
            path.append(router)
            print("Packet reached", router)

            if router == "R3" and dst.startswith("192.168.3."):
                break

            yield env.timeout(0.005)

            network = ".".join(dst.split(".")[:3]) + ".0/24"
            next_hop = routes[router].get(network)

            if next_hop is None:
                print("No route found! Packet dropped.")
                break

            router = next_hop

        if router and dst.startswith("192.168.3.") and router == "R3":
            print("Packet delivered to", dst)
            print("Routing path:", " -> ".join(path))


def client(env, channel, src, dst, router):
    yield env.timeout(0.1)
    print("\nClient sends packet:", src, "->", dst)

    packet = {
        "src": src,
        "dst": dst,
        "router": router
    }

    yield channel.put(packet)


env = simpy.Environment()
channel = simpy.Store(env)

env.process(server(env, channel))
env.process(client(env, channel,
```

"192.168.1.20", "192.168.3.20", "R1"))

env.run()

Go BACK

```
import socket

c = socket.socket()
c.connect(("127.0.0.1", 5003))

total = 8
window = 4
base = 1

while base <= total:
    end = min(base + window - 1, total)

    packets = " ".join(str(i) for i in range(base, end + 1))
    print("Sending:", packets)
    c.send(packets.encode())

    ack = int(c.recv(1024).decode())
    print("Received ACK:", ack)

    if ack >= base:
        base = ack + 1
    else:
        print("Retransmitting window")

c.close()
```

```
import socket

s = socket.socket()
s.bind(("127.0.0.1", 5003))
s.listen(1)

print("Receiver waiting...")
c, addr = s.accept()

expected = 1
lost_packet = 6
total = 8

while expected <= total:
    data = c.recv(1024).decode()

    if not data:
        break

    for x in data.split():
        packet = int(x)

        if packet == lost_packet:
            print("Packet", packet, "Lost")
            lost_packet = -1

        elif packet == expected:
            print("Received Packet:", packet)
            expected += 1

        elif packet < expected:
            print("Duplicate Packet:", packet)

        else:
            print("Discarded Packet:", packet)

    c.send(str(expected - 1).encode())

c.close()
s.close()
```

```
import simpy

def sender(env):
```

```
    total = 8
    window = 4
    base = 1
    lost_packet = 2

    while base <= total:
        end = min(base + window - 1, total)

        for i in range(base, end + 1):
            print(f"{env.now}: Sending Packet {i}")

        yield env.timeout(2)

        if base <= lost_packet <= end:
            print(f"{env.now}: Packet {lost_packet} Lost")
            print(f"{env.now}: ACK {base - 1} Received")
            print(f"{env.now}: Retransmitting from Packet {lost_packet}")

            for i in range(lost_packet, end + 1):
                print(f"{env.now}: Retransmitting Packet {i}")

            yield env.timeout(2)
            print(f"{env.now}: Received ACK {end}")

        else:
            print(f"{env.now}: Received ACK {end}")

        base = end + 1


env = simpy.Environment()
env.process(sender(env))
env.run()
```

Arp

```
import socket
import time

server = socket.socket()
server.bind(("127.0.0.1", 5000))
server.listen(1)

print("ARP Server started")

ip_table = {
    "192.168.1.10": "AA:BB:CC:DD:EE:FF",
    "192.168.1.20": "11:22:33:44:55:66"
}

arp_cache = {}
expiry = 10

while True:
    conn, addr = server.accept()
    print("Client connected")

    ip = conn.recv(1024).decode().strip()
    now = time.time()

    if ip in arp_cache and now < arp_cache[ip][1]:
        mac = arp_cache[ip][0]
        print("Cache hit")
    else:
        if ip in arp_cache:
            print("Cache expired")
            del arp_cache[ip]
        else:
            print("Cache miss")

        if ip in ip_table:
            mac = ip_table[ip]
            arp_cache[ip] = (mac, now + expiry)
```

```
        else:
            mac = "MAC address not found"

    conn.send(mac.encode())
    conn.close()


import socket

client = socket.socket()
client.connect(("127.0.0.1", 5000))

ip = input("Enter IP address: ")
client.send(ip.encode())

mac = client.recv(1024).decode()
print("MAC address:", mac)

client.close()


import simpy

receiver_ip = "10.10.1.20"
receiver_mac = "BB:BB:BB:BB:BB:20"
cache_timeout = 5
arp_cache = {}

def send_data(env):
    print(f"\nTime {env.now}: Checking ARP cache")

    if receiver_ip in arp_cache:
        mac, expiry = arp_cache[receiver_ip]

        if env.now < expiry:
            print("ARP Cache: HIT")
            print("Sending DATA to", mac)
            return
        else:
            print("ARP Cache: EXPIRED")
            del arp_cache[receiver_ip]
    else:
        print("ARP Cache: MISS")

    print("Host A: Broadcasting ARP Request")
    print("Who has", receiver_ip, "?")

    yield env.timeout(1)

    print("Host B: Sending ARP Reply")
    print(receiver_ip, "is at", receiver_mac)

    arp_cache[receiver_ip] = (receiver_mac, env.now + cache_timeout)

    print("ARP Cache Updated")
    print("Sending DATA to", receiver_mac)

def main(env):
    print("--- ARP Simulation Started ---")

    yield env.process(send_data(env))

    yield env.timeout(2)
    yield env.process(send_data(env))

    yield env.timeout(4)
    yield env.process(send_data(env))

    print("\n--- ARP Simulation Completed ---")
```

Sr

```
import simpy

def sender(env):
    total = 8
    window = 4
    base = 1
    lost_packet = 2

    while base <= total:
        end = min(base + window - 1, total)

        for i in range(base, end + 1):
            print(f"{env.now}: Sending Packet {i}")

        yield env.timeout(2)

        if base <= lost_packet <= end:
            print(f"{env.now}: Packet {lost_packet} Lost")
            print(f"{env.now}: ACK {base - 1} Received")
            print(f"{env.now}: Retransmitting from Packet {lost_packet}")

            for i in range(lost_packet, end + 1):
                print(f"{env.now}: Retransmitting Packet {i}")

            yield env.timeout(2)
            print(f"{env.now}: Received ACK {end}")

        else:
            print(f"{env.now}: Received ACK {end}")

        base = end + 1


env = simpy.Environment()
env.process(sender(env))
env.run()
```

```
import socket

s = socket.socket()
s.bind(("127.0.0.1", 5000))
s.listen(1)

print("Receiver waiting...")
c, addr = s.accept()

expected = 1
lost = 2
total = 8

while expected <= total:
    data = c.recv(1024).decode().strip()

    if not data:
        break

    for x in data.split():
        packet = int(x)

        if packet == lost:
            print("Packet", packet, "Lost")
            lost = -1
        else:
            print("Received Packet:", packet)
            c.sendall(("ACK " + str(packet) + "\n").encode())

c.close()
s.close()
```

```
import socket

c = socket.socket()
c.connect(("127.0.0.1", 5000))

total = 8
window = 4
base = 1

while base <= total:
    end = min(base + window - 1, total)

    for i in range(base, end + 1):
        print("Sending Packet:", i)
        c.sendall((str(i) + " ").encode())

    c.sendall(b"\n")

    acked = set()

    try:
        c.settimeout(1)

        for i in range(base, end + 1):
            data = c.recv(1024).decode()
            for x in data.split():
                if x.startswith("ACK"):
                    acked.add(int(x[3:]))

    except socket.timeout:
        pass

    for i in range(base, end + 1):
        if i not in acked:
            print("Retransmitting Packet:", i)
            c.sendall((str(i) + " ").encode())

    base = end + 1

c.close()
```

Csma

```
import simpy
import random

random.seed(1)

env = simpy.Environment()
channel = simpy.Store(env)

stats = {"success": 0, "collisions": 0}


def server(env, channel, stats):
    while True:
        first = yield channel.get()
        requests = [first]

        yield env.timeout(0.001)

        while channel.items:
            requests.append(channel.items.pop(0))

        if len(requests) > 1:
            stats["collisions"] += 1
            print("COLLISION! Multiple nodes transmitted together")

            for node, result in requests:
                result.succeed(False)
        else:
```

```
            node, result = requests[0]
            print(node, "channel IDLE - transmitting")

            yield env.timeout(0.01)

            print(node, "transmission successful")
            result.succeed(True)


def node(env, name):
    cw = 4

    for packet in range(1, 4):
        yield env.timeout(random.uniform(0.01, 0.05))
        print(name, "generated packet", packet)

        for attempt in range(5):
            result = env.event()
            yield channel.put((name, result))
            success = yield result

            if success:
                stats["success"] += 1
                cw = 4
                break

            if attempt < 4:
                backoff = random.randint(1, cw)
                print(name, "backoff:", backoff, "slots")

                yield env.timeout(backoff * 0.001)
                cw = min(cw * 2, 16)
                print(name, "new CW:", cw)
        else:
            print(name, "packet failed after retries")


env.process(server(env, channel, stats))

for i in range(1, 6):
    env.process(node(env, "N" + str(i)))

env.run()

print("\n--- Final Results ---")
print("Successful transmissions:", stats["success"])
print("Collision events:", stats["collisions"])
```

Tcp Multi

```
import socket
import threading

s = socket.socket()
s.bind(("localhost", 5001))
s.listen()

def chat(c):
    while True:
        msg = c.recv(1024).decode()
        print("Client :", msg)
        server_msg = input("Sever :" )
        c.send(("Server: " + server_msg).encode())

while True:
    c, addr = s.accept()
    threading.Thread(target=chat, args=(c,), daemon=True).start()


import socket

c = socket.socket()
c.connect(("localhost", 5000))
```

```
while True:
    msg = input("Enter message: ")
    c.send(msg.encode())
    print(c.recv(1024).decode())
```

Congestion control

```
import simpy
import random

random.seed(1)

env = simpy.Environment()
channel = simpy.Store(env)
ack = simpy.Store(env)

data = 10
cwnd = 1
loss = 0.1
rtt = 0.02


def receiver(env, channel, ack, loss, rtt):
    while True:
        packet = yield channel.get()

        if random.random() < loss:
            print("Packet lost:", packet)
        else:
            print("Receiver got packet:", packet)
            yield env.timeout(rtt)
            yield ack.put(packet)


def sender(env):
    global cwnd

    for i in range(1, data + 1):
        print("Send packet:", i, "cwnd:", cwnd)
        yield channel.put(i)

        yield env.timeout(rtt)

        if len(ack.items):
            yield ack.get()
            cwnd += 1
        else:
            cwnd = max(1, cwnd // 2)
            print("Loss detected; reduce cwnd")


env.process(receiver(env, channel, ack, loss, rtt))
env.process(sender(env))

env.run()


import socket
import random

random.seed(1)

s = socket.socket()
s.bind(("127.0.0.1", 5000))
s.listen(1)

print("Receiver waiting...")
c, addr = s.accept()

loss = 0.1
total = 10
```

```
while True:
    data = c.recv(1024).decode().strip()

    if not data:
        break

    for x in data.split():
        packet = int(x)

        if random.random() < loss:
            print("Packet lost:", packet)
        else:
            print("Receiver got packet:", packet)
            c.sendall(("ACK " + str(packet) + "\n").encode())

c.close()
s.close()
```

```
import socket

c = socket.socket()
c.connect(("127.0.0.1", 5000))
c.settimeout(1)

total = 10
cwnd = 1

for i in range(1, total + 1):
    print("Sending packet:", i, "cwnd:", cwnd)
    c.sendall((str(i) + "\n").encode())

    try:
        ack = c.recv(1024).decode().strip()
        print("Received:", ack)
        cwnd += 1

    except socket.timeout:
        cwnd = max(1, cwnd // 2)
        print("Packet loss detected; reduce cwnd to", cwnd)

c.close()
```

CRC

```
import socket

s = socket.socket()
s.bind(("localhost", 5001))
s.listen()

print("CRC Server Started")

c, addr = s.accept()

frame = c.recv(1024).decode()
print("Received Frame:", frame)

gen = input("Enter generator: ")

bits = list(frame)

for i in range(len(frame)-len(gen)+1):
    if bits[i] == '1':
        for j in range(len(gen)):
            bits[i+j] = str(int(bits[i+j]) ^ int(gen[j]))

r = ''.join(bits[-(len(gen)-1):])

if '1' in r:
    c.send(b"Error Detected")
```

```
else:
    c.send(b"No Error")

c.close()
s.close()
```

```
import socket

s = socket.socket()
s.connect(("localhost", 5001))

data = input("Enter data: ")
gen = input("Enter generator: ")

bits = list(data + "0" * (len(gen)-1))

for i in range(len(data)):
    if bits[i] == '1':
        for j in range(len(gen)):
            bits[i+j] = str(int(bits[i+j]) ^ int(gen[j]))

crc = ''.join(bits[-(len(gen)-1):])
frame = data + crc

print("CRC:", crc)
print("Frame:", frame)

s.send(frame.encode())

print("Server:", s.recv(1024).decode())

s.close()
```

```
import simpy

def crc_division(data, gen):
    bits = list(data)

    for i in range(len(data) - len(gen) + 1):
        if bits[i] == '1':
            for j in range(len(gen)):
                bits[i + j] = str(int(bits[i + j]) ^ int(gen[j]))

    return ''.join(bits[-(len(gen) - 1):])


def sender(env, channel):
    data = input("Enter data: ")
    gen = input("Enter generator: ")

    padded = data + "0" * (len(gen) - 1)
    crc = crc_division(padded, gen)
    frame = data + crc

    print("CRC:", crc)
    print("Frame:", frame)

    yield env.timeout(1)
    yield channel.put((frame, gen))


def receiver(env, channel):
    frame, gen = yield channel.get()

    print("Received Frame:", frame)

    yield env.timeout(1)

    remainder = crc_division(frame, gen)

    if '1' in remainder:
        print("Error Detected")
    else:
        print("No Error")
```

```
env = simpy.Environment()
channel = simpy.Store(env)

env.process(sender(env, channel))
env.process(receiver(env, channel))

env.run()
```